

Week3-作业

作业

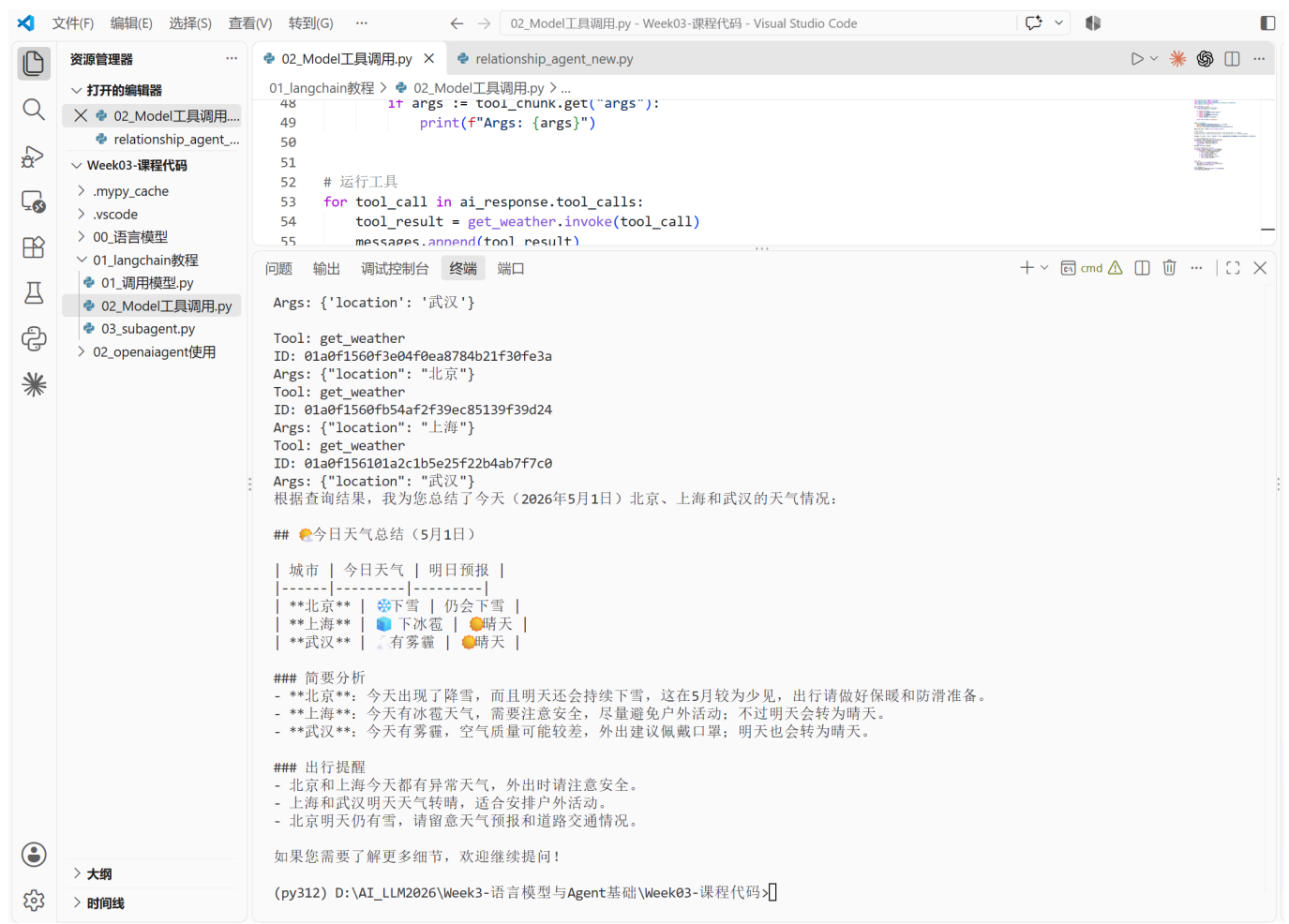
作业1: 安装langchain 和 openai-agent，运行 langchain / 02_Model工具调用.py

回答如下问题：

- 1、 langchain 工具调用 和 llm function call 有什么区别？
- 2、 langchain 工具调用 的速度是受到什么影响？

作业2: 阅读意图识别 01-intent-classify 代码，梳理源文件的作用，绘制从fastapi 接受请求到返回结果的流程，绘制流程图（手绘、自然语言表达）。

作业1



- 1. LLM Function Calling 是模型能力：根据用户问题，生成要调用的函数名称和参数，本身不执行函数。LangChain 工具调用是框架层的封装：描述工具、利用模型的 Function Calling、执行函数并回传结果。
- 2. 主要受五个因素影响：
 - 1. 模型响应速度：模型大小、服务负载、输入和输出长度。

2. 模型调用次数：通常需要先生成工具请求，再根据工具结果回答。
3. 工具执行耗时：本地计算、数据库查询或外部 API 的响应时间。
4. 网络延迟：访问模型服务和远程工具的连接速度。
5. 执行方式：多个工具串行执行通常比并行更慢。

作业2

用户发送请求 (JSON, 包含要分类的文本)

↓

main.py: FastAPI 根据请求地址, 选择对应的接口

↓

data_schema.py: 检查请求中的字段和数据类型是否符合要求

↓

main.py: 开始计时, 准备存放结果的响应对象

↓

logger.py: 提供日志功能, 用来记录请求信息

↓

调用对应的分类方法 (四选一)

|

├─ /regex → model/regex_rule.py

| 查看文本中有没有指定关键词 → 得到类别

| 关键词规则保存在 config.py

|

├─ /tfidf → model/tfidf_ml.py

| 把句子切成词 → 去掉停用词 → 把词转换成向量

| → 用训练好的分类器判断类别

| 使用模型文件: assets/weights/tfidf_ml.pkl

|

├─ /bert → model/bert.py

| 把文本转换成 BERT 能处理的向量 → 分批交给模型

| → 选出分数最高的类别

| 使用模型权重: assets/weights/bert.pt

| 类别名称保存在 config.py

|

└─ /gpt → model/prompt.py

| 找出与用户文本最相似的 10 条已标注例子

| → 把例子、可选类别和用户文本写进提示词

| → 发给远程大模型 → 取出模型回答的类别

| 例子来自: assets/dataset/dataset.csv

| 大模型的地址、名称等配置保存在 config.py

↓

main.py: 把执行结果放入响应对象

├─ 成功: 填写分类结果, error_msg = "ok"

└─ 失败：填写错误信息 error_msg
↓
main.py: 计算这次处理用了多长时间，填写 classify_time
↓
按照 data_schema.py 定义的格式返回结果
↓
FastAPI 把结果转换成 JSON → 返回给用户